BETTROI.

Team Collaboration SOP

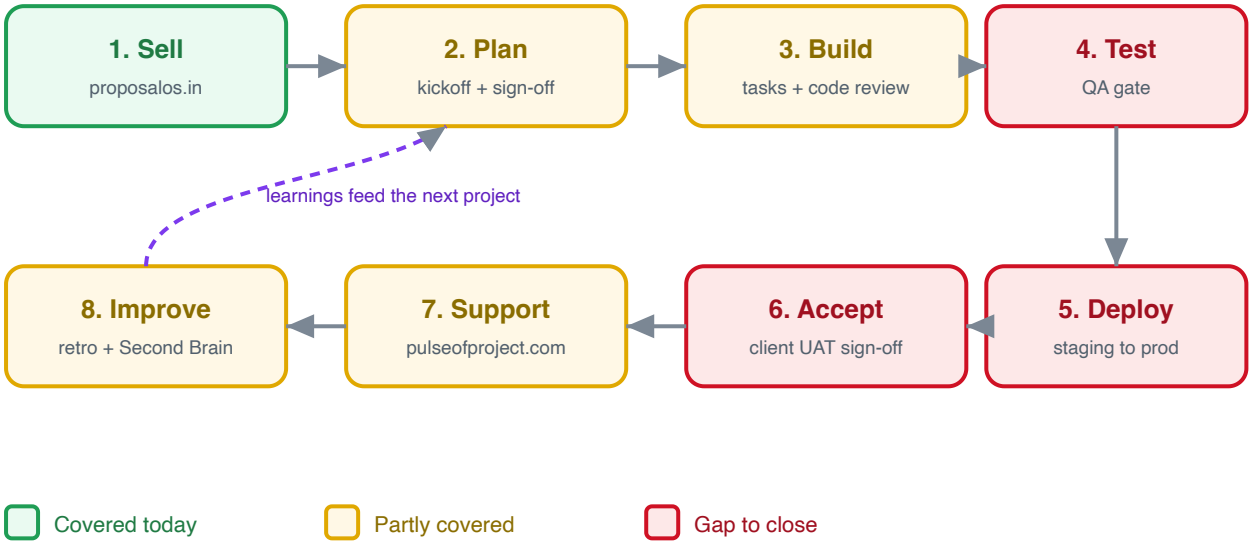
How the Bettroi software team works together, shares documents, prepares for client meetings, and delivers software end to end — with everything captured in the Second Brain (Bettroi Vault)

Standard Operating Procedure

Document ID	BETTROI-SOP-001
Version	2.6
Status	Approved
Classification	Internal / Confidential
Owner	cmd@hopehospital.com
Effective date	June 2026
Applies to	Bettroi / HopeTech software team
Canonical copy	corpus/playbook/07-team-collaboration.md (Bettroi Vault)

The full delivery lifecycle

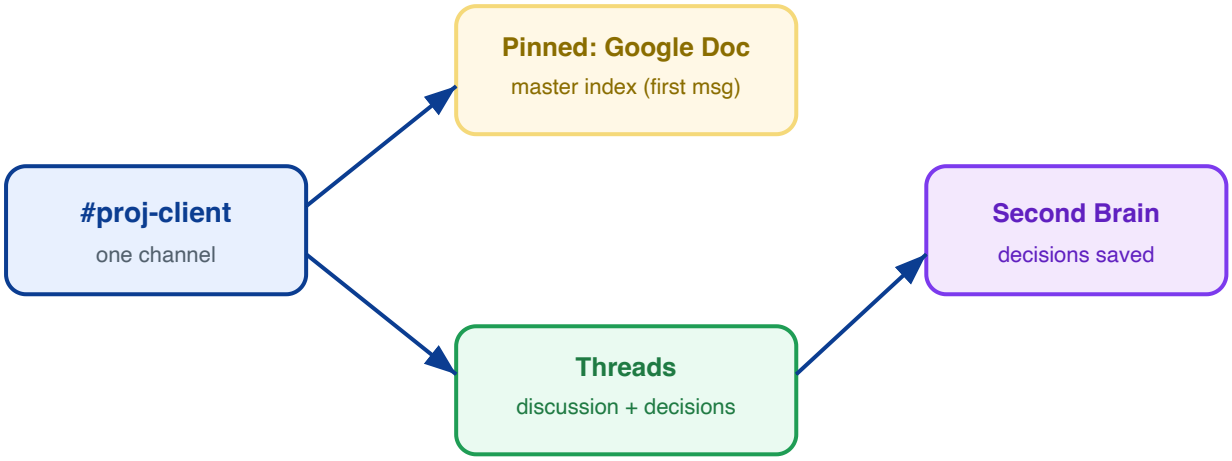
Every project moves through eight stages. Green stages we already run well; amber are partly covered; red are the gaps this SOP now closes.



Pages 1 to 5 cover how we communicate and keep documents straight (the foundation). Pages 6 to 12 add the delivery steps that turn this into a complete software workflow.

1 Slack is our single collaboration channel

All project conversation lives in Slack. Every project gets one channel, and its first pinned message is the Google Doc index.



One channel per project, named **#proj-client**. No scattered DMs for project work.

First pinned message = the Google Doc master index (see page 2). Always one click away.

Use threads for discussions. Key decisions get summarized back into the Second Brain.

External collaborators join via Slack Connect, never by sharing internal channels.

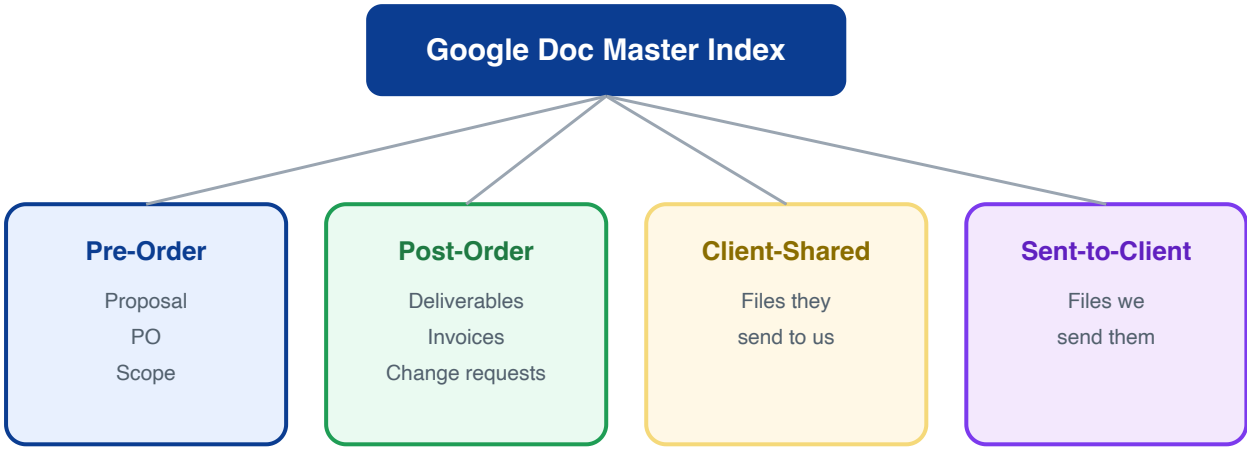
Email is for formal records only — all project info lives in Slack. If something arrives by email, post it (or its link) into the project thread.

Slack is how newcomers get context — a person joining mid-project reads the thread (Slack's summary gives the full history). Email and WhatsApp cannot.

One DRI / tech lead per project, from discovery; on join/leave there is a proper handoff so context is never lost.

2 The Google Doc master index

One living Google Doc per project holds hyperlinks to every document, organised by before vs after the order (and within each, received-from-client vs sent-to-client). It is the map to everything, and it is the description of the project's Slack thread.



Every shared file gets a **row in the index** AND a copy/link in the project folder (four sub-folders: before/after order × received/sent).

The index URL is the **pinned first Slack message** and is also recorded in the Second Brain.

Source of truth = the index + the vault, never someone's local Downloads folder.

Before the PO the project lives here; **once the PO arrives it moves into Pulse of Project** (plan, kickoff, and everything after).

3 The 4:00 PM pre-demo standup

One day before any demo or client meeting, the whole team meets at **4:00 PM IST** to confirm we are ready. Attendance is mandatory for everyone on the project.

Readiness checklist (run every time):

- Demo script ready and rehearsed
- Environment and test data verified working
- Open bugs triaged; blockers flagged
- An owner assigned to each talking point
- Fallback plan if something breaks live

The standup is auto-captured: it appears in the calendar feed, and if recorded, as a meeting note in the project folder.

Meeting discipline & client etiquette

The 4 PM standup is for demos. Beyond it, every single client meeting gets an internal pre-brief first — owned by the Project Manager.

Internal pre-brief before EVERY client meeting (not just demos). The **Project Manager fixes the internal meeting before the client meeting** — who attends, what we will demo, the sequence, the expected outcome. **No internal pre-brief → the client call is not attended.**

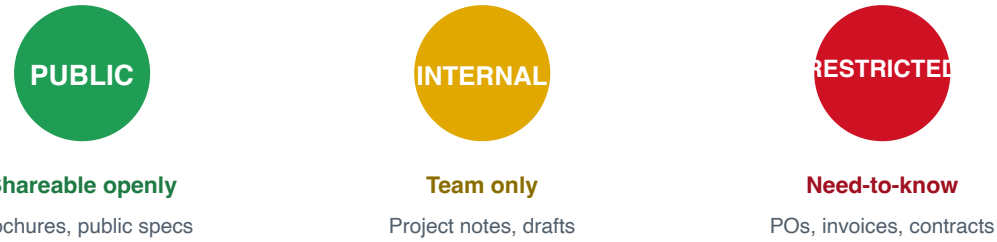
Notify the client of attendees ahead of each meeting. For enterprise/government clients (who expose their data to us), the PM tells them in advance who from our team will join.

Inform the client when a team member joins or leaves the project. New people are never introduced into a client's system or call without the client knowing first.

Why: we are distributed across geographies. A short internal alignment makes us look like one professional team, prevents surprises in front of the client, and respects the data access enterprise clients have granted us.

4 File and document access tiers

Every document carries a sensitivity tier. This decides where it can be shared and stored.



Shareable openly
Brochures, public specs

Team only
Project notes, drafts

Need-to-know
POs, invoices, contracts

Public — freely shareable material.

Internal — project notes and drafts; team only, never in public channels.

Restricted — anything with pricing, money, or PII: POs, invoices, contracts.

PHI is never committed to the shared vault. It stays in the gitignored scratch space.

Naming: `YYYY-MM-DD - client - doc-type - short-title`

5 Bug tracking and who does what

Bug tracking and the who-does-what view live in **pulseofproject.com** (POP), connected to the Second Brain vault by API (synced daily into `brain_chunks` with severity P1/P2/P3, status, and assignee). Each bug links back to its Slack thread and its row in the Google Doc index.

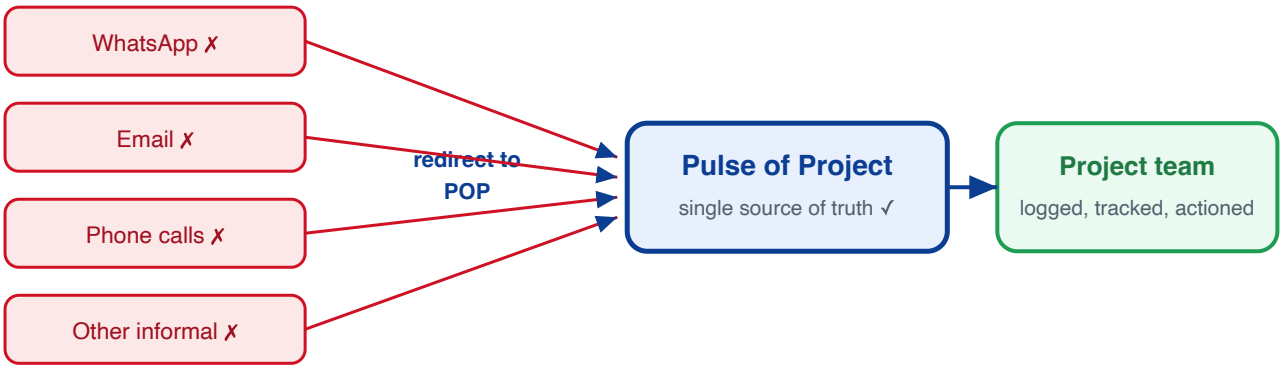
Activity	Owner	Where it lives
Project conversation	Whole team	Slack #proj-client
Document links (pre and post order)	Project lead	Google Doc master index
Pre-demo readiness	Whole team @ 4 PM	Standup + checklist
Client bugs and who-does-what	Assigned dev	pulseofproject.com (API to vault)
Restricted docs (PO, invoice)	Project lead	Restricted tier, not public

Source of truth: the Google Doc index plus the Second Brain vault. pulseofproject.com syncs bug and task status into the vault by API.

KEY POLICY

All client feedback goes through Pulse of Project (POP)

POP is the official and **only** channel for the client to log feedback. This is what makes the project trackable.



POP is the only channel for the client to submit feedback on deliverables, milestone submissions, UAT, bug reports, change requests (CRs), and enhancement requests.

Feedback that arrives via **WhatsApp, email, phone, or any informal channel is redirected to POP** by the team, so it is documented and tracked.

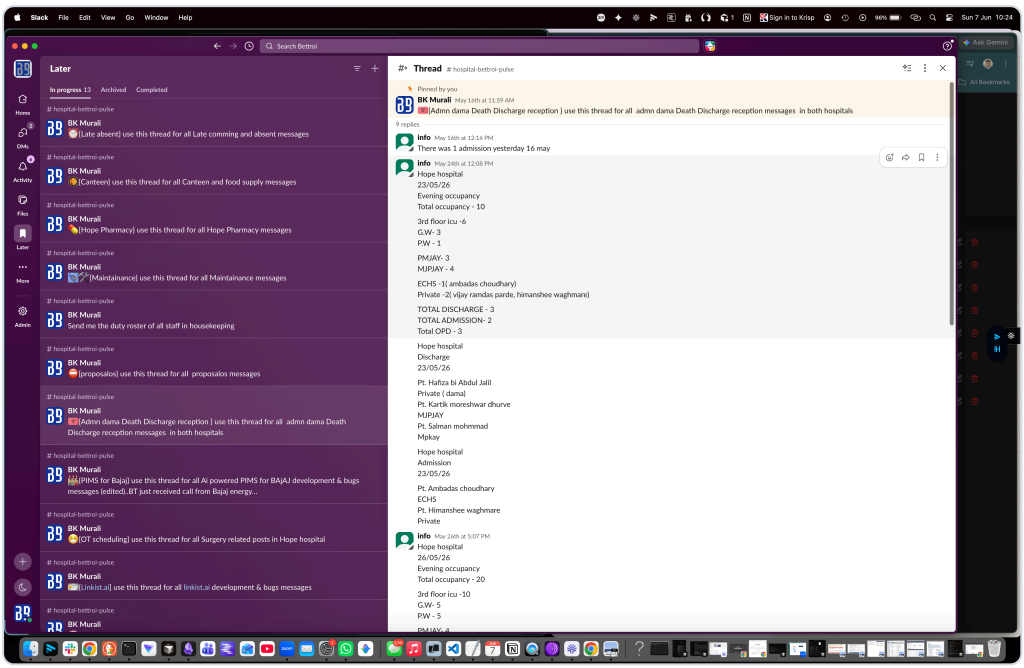
If it is not in POP, it is **not on the official list** the team works from.

UAT feedback goes straight into POP — give the client the POP URL and have them log it there directly, not in WhatsApp, Drive, or a Word doc someone then has to chase down.

Why this matters: a lot of productive time is currently lost extracting feedback from scattered channels and consolidating it for the team. Making POP the single source of truth improves **accountability, traceability, and overall efficiency**. POP enhancements to better support this process are in progress (discussed with Sahil).

Slack thread directory — internal dev coordination

Each project has one dedicated Slack thread for development & bugs, bookmarked in **Later** so it is always one click away. One thread per topic keeps internal coordination from scattering.



Our Slack **Later** view: one bookmarked thread per topic, each pinned "use this thread for all … messages".

Thread	Use it for
Bajaj PIMS — dev & bugs	All AI-powered PIMS for Bajaj development & bug messages
Hope — dev & bugs	All Hope product development & bug messages
<new project> — dev & bugs	Pattern: one "development & bugs" thread per project, pinned "use this thread for all <project> development & bugs messages"

> **One thread per project** for dev & bug coordination; bookmark it in Slack **Later**.

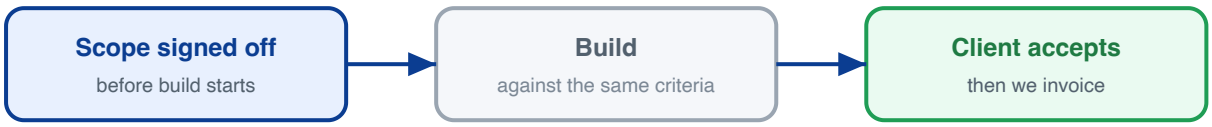
> Pin the thread with: *"use this thread for all <project> development & bugs messages"*.

> These threads are for **internal team coordination**. Official client feedback still goes through **Pulse of Project (POP)** — see the POP policy.

Note: this directory was set up recently as part of our internal collaboration. Keep it current — add a new dev & bugs thread whenever a new project starts.

6 Requirements and acceptance sign-off

Two written gates wrap every order: the client agrees the scope before we build, and confirms it works before we invoice. Same checklist, checked twice.



Acceptance criteria are written into the scope doc (in proposalos.in / Google Doc index) and the client signs it before work starts.

Before kickoff: prepare a project plan + delivery plan, walk the client through both, get sign-off — then build. Prevents scope creep (the "Limitless Brain" lesson, where unmanaged changes doubled/tripled cost).

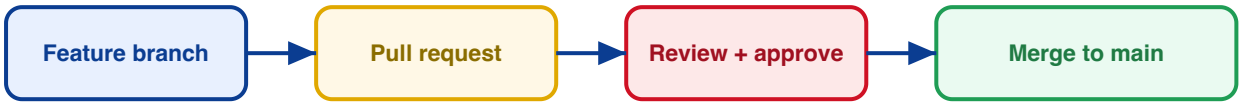
The client must provide a clear wireframe of each page they want developed, before the build starts. Developers build to an agreed visual, not an assumption — removing rework and back-and-forth.

Change requests go through the same sign-off, never verbally; each is logged in the Google Doc index (and POP after the PO).

Acceptance test at delivery: client confirms each criterion is met. That sign-off triggers the invoice.

7 Code review and Git workflow

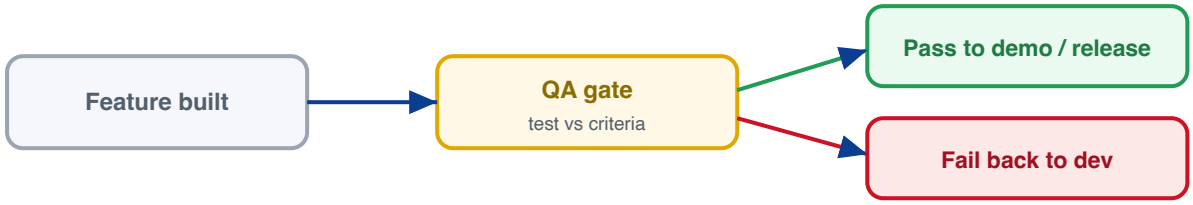
No code reaches the main branch without review. This is where most bugs are caught before a client ever sees them.



- **Branch per task**; never commit straight to main.
- **Pull request reviewed by at least one other developer** before merge.
- **No hardcoded secrets**; credentials come from environment variables only.
- Each PR links back to its **pulseofproject.com** task.

8 Testing and the QA gate

A feature is tested against its acceptance criteria before any demo or release. Testing must not happen live in front of the client.



A **dedicated QA (not the developer) runs manual + regression testing** against the acceptance criteria before anything passes to staging.

Bugs found are logged in **pulseofproject.com** and fixed before the gate is passed.

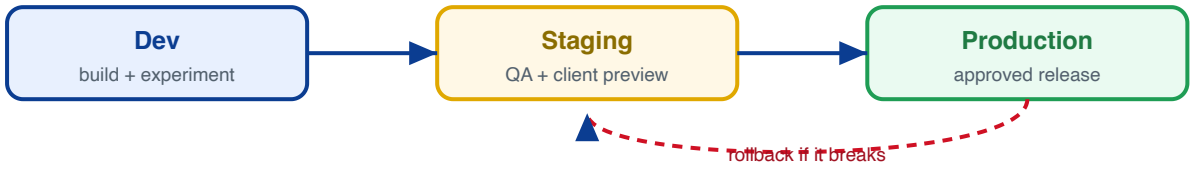
Reopened bugs get an audit — when a closed bug comes back, cross-check why (bad fix, or QA miss).

Estimates include QA time + a buffer. AI makes dev and QA faster (a day of QA can take 2–3 hours), so the buffer stays small — but it is always there.

The **4 PM pre-demo standup** confirms the QA gate is green for everything being shown.

9 Deploy, environments and rollback

Changes flow through fixed environments. Every release has an approver and a way to undo it.



Dev to Staging to Production; never edit production directly.

Staging/UAT uses masked production data — never the real production data. The client tests on staging; only after sign-off does it go live.

Every project needs a staging environment (e.g. Linguist currently has none — fix that); bugs can't be safely reproduced without one.

Client-facing releases need a **named approver**.

Every release has a **rollback plan** so a bad deploy can be reverted fast.

10 Incidents, support and escalation

When something breaks for a client in production, response time depends on how serious it is.

Severity	Example	Response
P1 Critical	System down, client cannot work	Immediate; all-hands hotfix
P2 High	Major feature broken, workaround exists	Same business day
P3 Normal	Minor bug, low impact	Next planned cycle

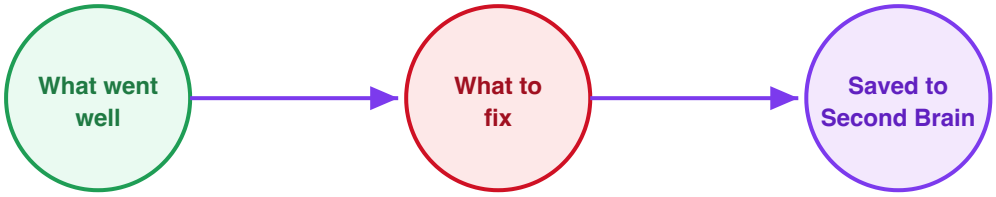
> All incidents are logged in **pulseofproject.com** and announced in the project Slack channel.

> P1 fixes follow the same **review and rollback** rules; speed never skips review.

> Each incident gets a one-line **root-cause note** saved to the Second Brain.

11 Retrospective and continuous improvement

After each project or major milestone, a short look-back so the next project runs better.



> Short retro after each project or milestone: **what went well, what to fix, one action each**.

> Lessons are saved to the **Second Brain** (and harvested as reusable patterns) so they compound.

12 Cross-cutting essentials

Rules that apply across every stage above.

NDA / NCNDA for every contributor — everyone who touches a project signs an NDA/NCNDA in their **personal capacity**, including part-time and external resources. They inadvertently see client and confidential data. **No access until signed.**

Access and secrets — new team members are granted repo / server / client-system access on joining and have it **revoked when they leave**. Credentials live in a secrets manager, never in chat or code.

Time and effort tracking — work is logged against the order so we can **bill accurately against the PO** in proposalos.in.

Definition of Ready / Definition of Done — a task only starts when it has clear acceptance criteria (Ready), and is only closed when built, reviewed, tested, and deployed (Done).

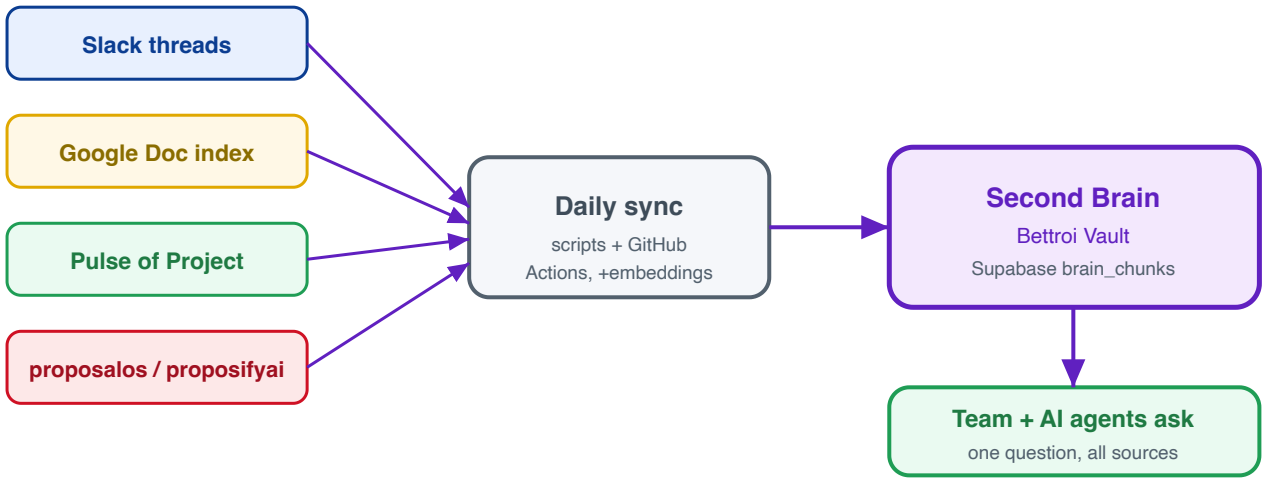
Process standard — this SOP is the process; it is being aligned to a recognised standard (ISO certification in progress).

The one-line summary: Communicate in Slack, keep every link in the Google Doc index, sign off scope before and after, review and test before you ship, deploy safely, and write down what you learned.

ADDENDUM

How it all lands in the Second Brain (Bettroi Vault)

Every collaboration tool feeds one place: the Bettroi Vault. Daily syncs pull each source into **brain_chunks** (a Supabase vector store), so the whole team and our AI agents can ask one question and get answers across all of it.



Source	How it is stored in the Second Brain
Slack (dev/bug threads)	Thread messages pulled via the Slack API into brain_chunks (searchable)
Google Doc master index	The index link is recorded in the vault; document links travel with each project
Pulse of Project	Bugs + projects (status, severity, who-does-what) synced into brain_chunks daily
proposalos / proposifyai	Proposals as notes; POs & invoices into brain_chunks marked **restricted** (off-limits)
This SOP + all vault notes	Markdown synced into brain_chunks, so the rules themselves are queryable

The payoff: nobody has to hunt across Slack, Docs, POP, and proposalos to reconstruct what happened. Ask the Brain once — "what P1 bugs are open for Bajaj and who owns them", "what did we quote Linkist" — and the answer is grounded in all of it. Financial data stays restricted.